

Master Informatique spécialité Intelligence Artificielle

Approches centrées sur les données

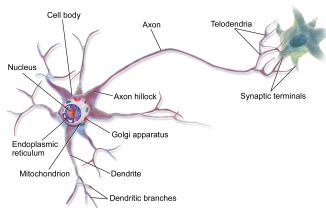
5 septembre 2024

Le Perceptron



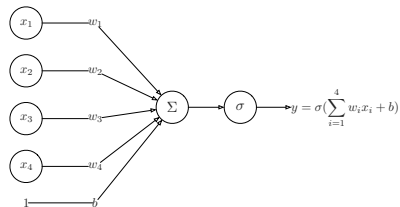
Inspiré d'un modèle biologique:

- Une unité de traitement de l'information
- Des stimulations en entrées qui sont traitées
- Des sorties communiquées à d'autres neurones ou organes

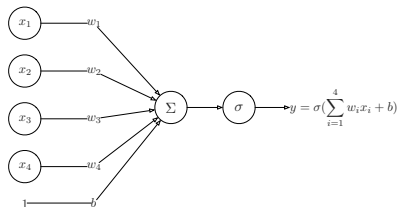


Un système artificiel extrêmement simplifié

- Traitement: somme et fonction non-linéaire
- Multiples entrées
- Une seule valeur de sortie



Définitions



- (x_1, x_2, x_3, x_4) est une **entrée** multi-dimensionnelle
- (w_1, w_2, w_3, w_4) sont les **poids** du modèle
- b est le **biais**
- σ est la **fonction d'activation**
- y est la **sortie**

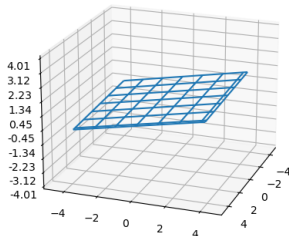
La fonction d'activation d'un perceptron est fonction échelon unité qui prend la valeur 1 lorsque la somme de entrées pondérées par leur poids respectif dépasse 0 et prend la valeur 0 sinon

Le Perceptron



Visualisation

La partie linéaire du perceptron permet d'approximer une fonction linéaire des données d'entrée. Dans le cas d'un perceptron à 2 entrées, la fonction linéaire s'annule sur un hyperplan (une surface plane) qui divise l'espace en deux parties.

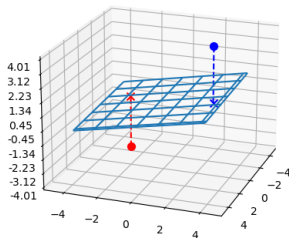


Le Perceptron



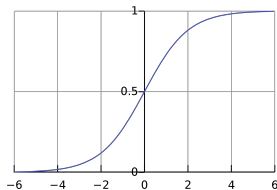
Classification avec un perceptron

L'ajout d'une fonction d'activation de type *échelon* permet, pour une entrée $A = (\alpha_0, \alpha_1)$ d'obtenir en sortie du perceptron une valeur binaire $\{0; 1\}$ selon le signe de $func(\alpha_0, \alpha_1)$

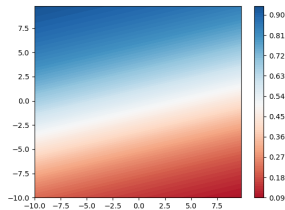


Du Perceptron au Neurone

- Le perceptron est limitée car il ne modélise que des fonctions linéaires
- L'ajout d'une fonction non-linéaire permet d'augmenter les utilisation du perceptron.



(a) Fonction sigmoïde.



(b) Exemples de fonctions sigmoïde.

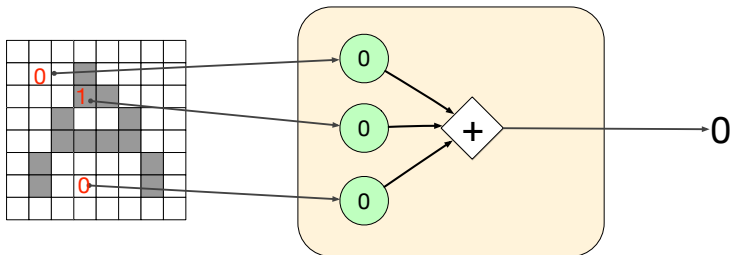
Pour chaque point du plan, le neurone renvoie une valeur entre 0 et 1 qui peut être utilisée pour approximer une probabilité.

Entraînement d'un neurone pour la classification



Principe pour de la classification d'images

- Deux catégories d'images: A et B
- Un neurone dont chaque entrée est la valeur (0 ou 1) d'un pixel
- Pas de biais
- Poids initialisés au hasard

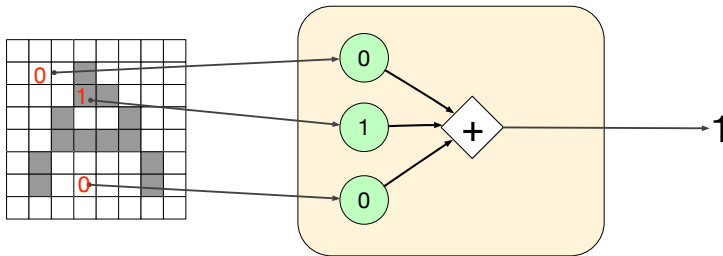


Entraînement d'un neurone pour la classification

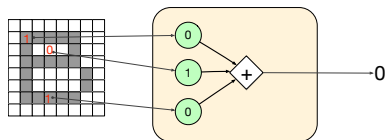


Principe pour de la classification d'images

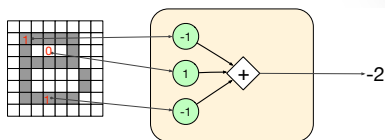
- Initialisation aléatoire des poids
- Calcul des sorties
- Modification des poids pour réduire l'erreur



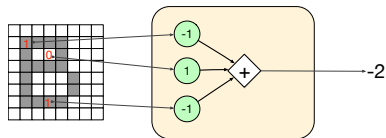
Entraînement d'un neurone pour la classification



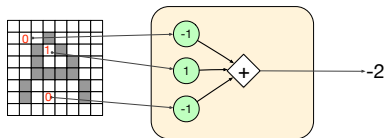
(1)



(2)



(3)



(4)

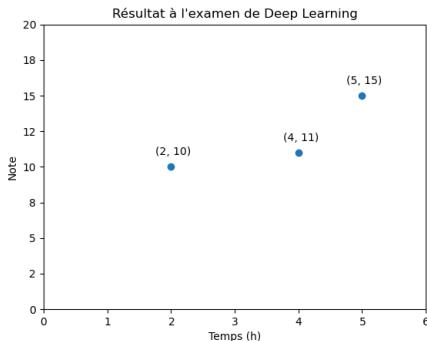


Algorithme de descente de gradient

Entraînement d'un neurone



- Nous souhaitons prédire la note sur 20 en fonction du temps de travail
- Données d'apprentissage: collectées auprès de 3 étudiants



Entraînement d'un neurone



1^{re} étape: choisir un modèle

- Nous supposons que la note est une fonction linéaire du temps de travail
- Selon cette hypothèse (qui n'est que pure spéculation), la note obtenue vérifierait l'équation suivante:

$$Note = Pente \times t + Connaissance \quad (1)$$

La note obtenue ne dépendrait que de deux **paramètres** de la fonction:

- la *Connaissance*, qui correspond à la note obtenue sans travail
- la *Pente*

Entraînement d'un neurone



2^e étape: mesurer l'erreur

- Nous effectuons une tâche de régression
- L'erreur commise est donc une "différence" entre la valeur prédite et la valeur que nous aurions du avoir, la valeur réelle.
- L'erreur peut être mesurée:
 - en valeur absolue
 - en valeur quadratique
 - etc.
- Pour cet exemple, choisissons une différence quadratique.

Entraînement d'un neurone



Définition

En apprentissage automatique, l'erreur est une fonction des paramètres du modèle et du jeu de données d'apprentissage est appelée **fonction de coût** (*loss function*). Notre tâche consiste à **estimer les paramètres** de notre modèle afin de **minimiser la fonction de coût**.

Entraînement d'un neurone



Simplification pour l'exemple

- Notre modèle nécessite de fixer 2 valeurs: *Connaissance* et *Pente*
- Pour simplifier l'exemple nous considérons que:

$$Pente = 1,5 \quad (2)$$

- Il nous reste à estimer *Connaissance*

$$Note = 1,5 \times t + Connaissance \quad (3)$$

Entraînement d'un neurone



Fonction de coût

La fonction choisie mesure l'erreur sur un ensemble d'exemple selon la formule suivante:

$$Erreur(Connaissance, (t_i, Note_i)_{i=1,2,3}) = \sum_i (Note_i - (1,5 \times t_i + Connaissance)) \quad (4)$$

Entraînement d'un neurone

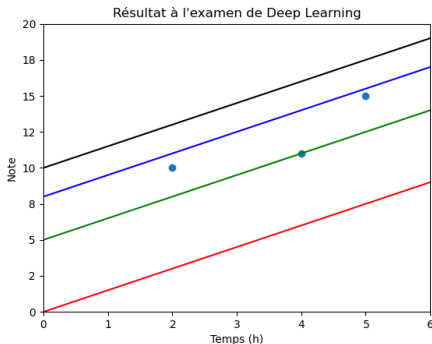


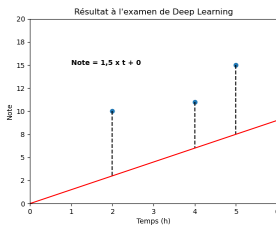
Figure 2: La descente de gradient est utilisé dans cet exemple pour déterminer la meilleure ordonnée à l'origine pour notre modèle linéaire. Les différentes figures correspondent à différents modèles obtenus pour différentes valeurs du paramètre *Connaissance*.

Entraînement d'un neurone



Initialisation: fixons aléatoirement la valeur du paramètre à estimer.

- $Pente = 1,5$
- $Connaissance = 0$



L'erreur quadratique obtenue est alors:

$$Erreur(Connaissance, data) = (10 - 1,5 \times 2 + 0)^2 \quad (5)$$

$$+ (11 - 1,5 \times 4 + 0)^2 \quad (6)$$

$$+ (15 - 1,5 \times 5 + 0)^2 \quad (7)$$

$$Erreur(Connaissance, data) = 130,5 \quad (8)$$

Entraînement d'un neurone



On pourrait simplement trouver la meilleure valeur en dérivant l'erreur mais pas dans un cas réel avec des millions de paramètres.
Nous allons donc utiliser l'algorithme de descente de gradient pour se rapprocher du meilleur modèle de façon itérative.

Entraînement d'un neurone

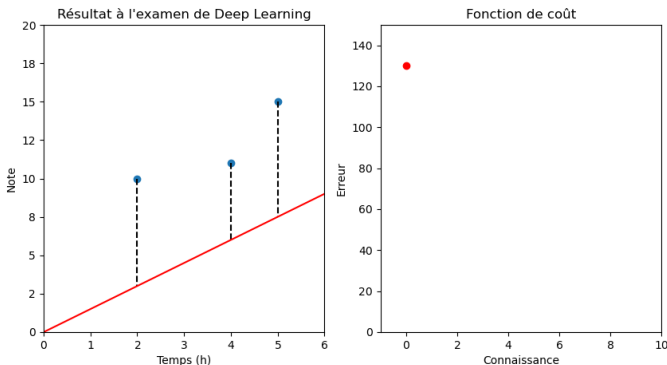


Figure 3: Le modèle linéaire est initialisé avec $Connaissance = 0$. L'erreur quadratique est reportée sur le graphe de droite en fonction de la valeur du paramètre $Connaissance$.

Entraînement d'un neurone



Observons la valeur de l'erreur quadratique pour différentes valeurs du paramètre *Connaissance*.

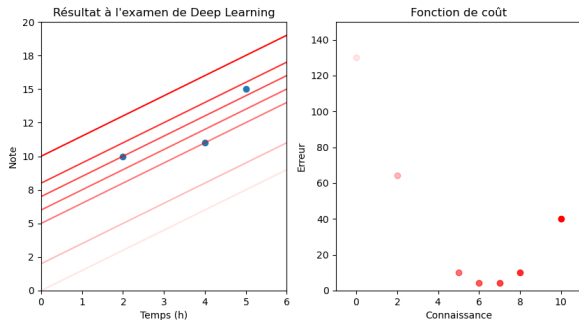


Figure 4: Le modèle linéaire est initialisé avec $Connaissance = 0$. L'erreur quadratique est reportée sur le graphe de droite en fonction de la valeur du paramètre *Connaissance*.

Entraînement d'un neurone



L'algorithme de descente du gradient:

- est utilisable pour un grand nombre de paramètres
- converge vers un minimum de la fonction de coût (même si ce minimum peut être local)
- calcule de plus en plus de valeurs lorsqu'il se rapproche de l'optimal : l'algorithme de descente de gradient optimise les paramètres en effectuant de grands pas lorsqu'il est loin de l'optimal et de petits pas lorsqu'il s'en rapproche.

Algorithme de descente de gradient



Consiste à dériver la fonction de coût par rapport au paramètre à optimiser:

$$Erreur(Connaissance, (t_i, Note_i)_{i=1,2,3}) = \sum_i (Note_i - (1,5 \times t_i + Connaissance))^2$$

(9)

Algorithme de descente de gradient



La fonction de coût par rapport au paramètre à optimiser peut s'écrire:

$$Erreur(Connaissance) = (10 - (1,5 * 2 + Connaissance))^2 \quad (10)$$

$$+ (11 - (1,5 * 4 + Connaissance))^2 \quad (11)$$

$$+ (15 - (1,5 * 5 + Connaissance))^2 \quad (12)$$

Algorithme de descente de gradient



La dérivée de la fonction de coût par rapport au paramètre à optimiser est:

$$\frac{dErreur(Connaissance)}{dConnaissance} = 2 \times (10 - (1,5 * 2 + Connaissance))(-1) \quad (13)$$

$$+ 2 \times (11 - (1,5 * 4 + Connaissance))(-1) \quad (14)$$

$$+ 2 \times (15 - (1,5 * 5 + Connaissance))(-1) \quad (15)$$

Algorithme de descente de gradient



À l'initialisation du modèle:

$$Connaissance = 0 \quad (16)$$

et donc

$$\frac{dErreur}{dConnaissance} = 6 \times Connaissance - 39 \quad (17)$$



Algorithme de descente de gradient

La dérivée, c-à-d le gradient de la fonction d'erreur par rapport au paramètre à optimiser est utilisée pour déterminer la nouvelle valeur du paramètre comme suit:

$$Connaissance = Connaissance - \frac{dloss}{dConnaissance} \quad (18)$$

En pratique la mise à jour est trop importante, on va la pondérer par un facteur appelé: *learning rate*.

$$Connaissance = Connaissance - \frac{dloss}{dConnaissance} \times lr \quad (19)$$

Algorithme de descente de gradient



Avec les valeurs à l'initialisation:

$$\textit{Stepsize} = \frac{d\textit{loss}}{d\textit{Connaissance}} \times lr \quad (20)$$

$$= -39 \times 0.05 \quad (21)$$

$$= -1,95 \quad (22)$$

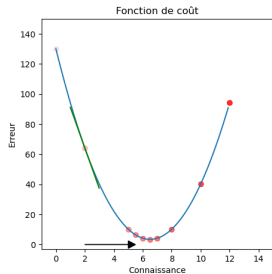
Et ainsi:

$$\textit{Connaissance} = \textit{Connaissance} - \frac{d\textit{loss}}{d\textit{Connaissance}} \times lr \quad (23)$$

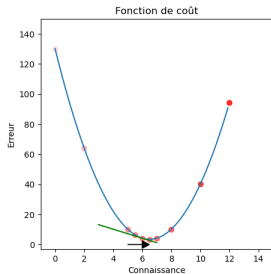
$$= 0 - (-39) \times 0.05 \quad (24)$$

$$= 1,95 \quad (25)$$

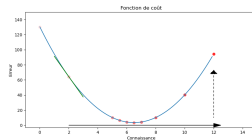
Algorithme de descente de gradient



(a) Tangente pour
Connaissance = 2



(b) Tangente pour
Connaissance = 5



(c) Tangente pour
Connaissance = 2

Figure 5: Fonction de coût en fonction de la valeur du paramètre *Connaissance*. Chaque sous-figure montre la tangente à cette fonction en un point différent et illustre la taille du pas nécessaire à la mise à jour du paramètre *Connaissance*. Si le pas est trop grand (figure de droite) la valeur de la loss ne décroît pas.

Algorithme de descente de gradient



Note:

Dans ce cas, simple, il suffit de déterminer le point pour lequel la pente de la tangente est nulle. La descente de gradient va, elle, permettre de converger vers ce point. C'est utile dans le cas où la fonction est trop complexe pour trouver le point où sa dérivée s'annule.

Algorithme de descente de gradient



Une fois la nouvelle valeur du paramètre obtenue, on recalcule l'erreur et son gradient pour faire une nouvelle mise à jour.

Algorithme de descente de gradient: itérations



Étape	Pas	Nouvelle valeur de Connaissance
0	1,95	$0 + 1,95 = 1,95$
1	1,365	$1,95 + 1,365 = 3,315$
2	0.9555	$3,315 + 0,9555 = 4,2705$
3	0.66885	$4,2705 + 0,66885 = 4,93935$
4	0,468195	$4,93935 + 0,468195 = 5,407545$
5	0,3277365	$5,407545 + 0,3277365 = 5,7352815$
6	0.229415549	$5,7352815 + 0.229415549 = 5.96469705$
7	0.160590	$5.96469705 + 0.160590 = 6.1252879$
8	0.11241363	$6.1252879 + 0.11241363 = 6.23770153$
9	0.078689	$6.23770153 + 0.078689 = 6.316391$
10	0.055082	$6.316391 + 0.055082 = 6.3714737$
11	0.0385578	$6.3714737 + 0.0385578 = 6.410031$

Algorithme de descente de gradient



Le pas de mise à jour diminue lorsqu'on se rapproche du minimum.

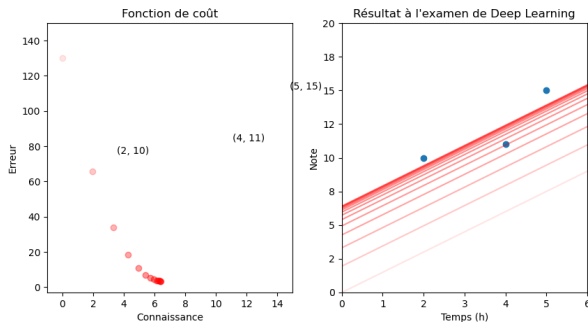


Figure 6: Valeurs obtenus par la méthode de descente de gradient. La figure de droite montre l'évolution de la fonction de coût au cours des itérations et la figure de droite montre les droites obtenues pour chacune des itérations.

Algorithme de descente de gradient



Quand s'arrêter?

Nombre d'itérations maximum il suffit de fixer un nombre d'étapes à ne pas dépasser

Fixer un pas minimum une fois que le pas devient plus petit qu'un certain seuil, la mise à jour est insignifiante et on arrête l'algorithme

Algorithme de descente de gradient



La méthode de descente de gradient s'articule en 8 étapes

1. Choisir un modèle
2. Choisir une fonction de coût
3. Initialiser le paramètre
4. Dériver la fonction de loss par rapport au paramètre à estimer
5. Calculer la dérivé pour le modèle actuel
6. Utiliser cette dérivée pour calculer le pas de mise à jour
7. Mettre à jour la valeur du paramètre avec le pas
8. Boucler à l'étape 5 jusqu'à atteindre un critère d'arrêt



Algorithme de descente de gradient pour plusieurs paramètres

Algorithme de descente de gradient

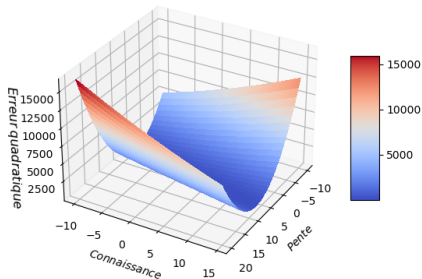


La fonction de coût devient une fonction à 2 paramètres:

$$Erreur(Connaissance, Pente, data) = (10 - (Pente \times 2 + Connaissance))^2 \quad (26)$$

$$+ (11 - (Pente \times 4 + Connaissance))^2 \quad (27)$$

$$+ (15 - (Pente \times 5 + Connaissance))^2 \quad (28)$$





Algorithme de descente de gradient

La fonction de coût devient une fonction à 2 paramètres:

$$\frac{\partial loss}{\partial Connaissance} = \frac{\partial}{\partial Connaissance} (10 - (Pente \times 2 + Connaissance))^2 \quad (29)$$

$$+ \frac{\partial}{\partial Connaissance} (11 - (Pente \times 4 + Connaissance))^2 \quad (30)$$

$$+ \frac{\partial}{\partial Connaissance} (15 - (Pente \times 5 + Connaissance))^2 \quad (31)$$

$$\frac{\partial loss}{\partial Connaissance} = 22 \times Pente + 6 \times Connaissance - 72 \quad (32)$$

On calcule également la dérivée partielle par rapport au second paramètre:

$$\frac{\partial loss}{\partial Pente} = 90 \times Pente + 22 \times Connaissance - 278 \quad (33)$$



Algorithme de descente de gradient

Comme précédemment nous initialisons le modèle aléatoirement:

$$\begin{cases} Pente = 1 \\ Connaissance = 0 \end{cases}$$

En utilisant le gradient de la fonction de coût et les paramètres initiaux du modèle nous pouvons calculer les gradients selon *Connaissance* et *Pente*:

$$\begin{cases} \frac{\partial loss}{\partial Pente}(0, 1) = -188 \\ \frac{\partial loss}{\partial Connaissance}(0, 1) = -50 \end{cases}$$



Algorithme de descente de gradient

Ainsi nous pouvons calculer les nouvelles valeurs de *Pente* et *Connaissance*:

$$\begin{cases} Pente = Pente - \frac{\partial \text{loss}}{\partial Pente}(0, 1) * lr \\ Connaissance = Connaissance - \frac{\partial \text{loss}}{\partial Connaissance}(0, 1) * lr \end{cases}$$

Si on fixe $lr = 0.05$ le pas est trop grand, on peut réduire la valeur de lr à 0.01. Remarquez que l'algorithme est très sensible au learning rate.

$$\begin{cases} Pente = 1 - (-188) * 0.01 \\ Connaissance = 0 - (-50) * 0.01 \end{cases}$$

$$\begin{cases} Pente = 2.88 \\ Connaissance = 0.5 \end{cases}$$

Visualisation

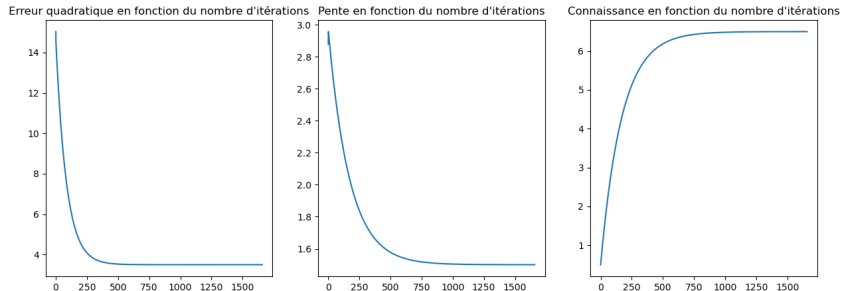
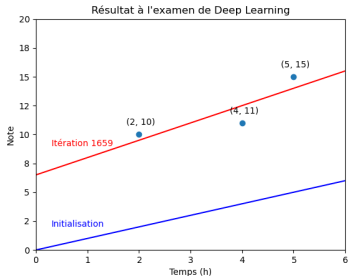
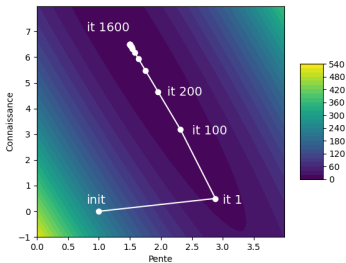


Figure 7: Évolution de la fonction de coût et des deux paramètres *Pente* et *Connaissance* au cours des itérations

Visualisation



(a) Modèle linéaire à l'initialisation et après 1659 itérations.



(b) Évolution des paramètres du modèle linéaire

Figure 8: Résultat de la recherche de paramètres avec une descente de gradient.

À retenir



- Le choix du modèle est essentiel, il s'agit d'un méta-paramètre
- Le learning rate et la fonction de coût sont également des méta-paramètres qu'il vous faut fixer
- Le learning rate est sensible
- Les critères d'arrêt sont également des méta-paramètres
- Il existe de nombreuses fonctions de coût qui peuvent être utilisées, cependant il est obligatoire que la fonction que vous utilisez soit dérivable.
- la descente de gradient nécessite que la fonction de coût soit continuellement dérivable.

Lien avec les réseaux de neurones

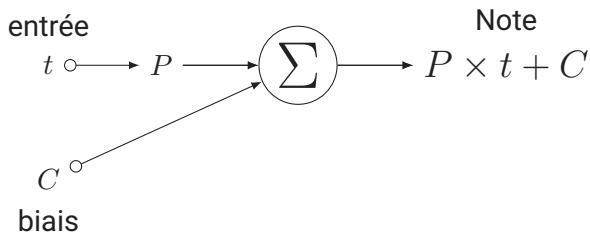


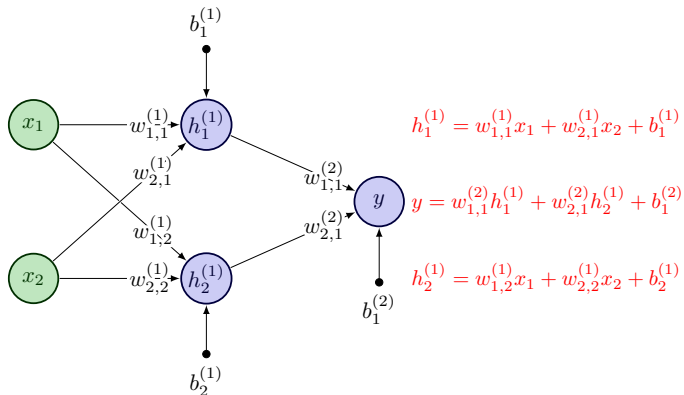
Figure 9: Régression Linéaire mise sous forme d'un réseau de neurones linéaire.

En TD nous verrons comment utiliser la descente de gradient avec plusieurs neurones et des fonctions d'activation.



Descente de Gradient Stochastique

Cas plus concret d'un réseau de neurones simple



$$Erreur(data, \Theta) = \sum (y - \hat{y}(d))^2$$



Cas plus concret d'un réseau de neurones simple

Jeu de données *data* : deux échantillons

- $(0.5, 0.2)$ pour lequel on observe la valeur 4
- $(1.1, 1)$ pour lequel on observe 3

Cas plus concret d'un réseau de neurones simple



Initialisation aléatoire des paramètres:

$$w_{1,1}^{(1)} = 0.1$$

$$w_{1,2}^{(1)} = 0.2$$

$$w_{2,1}^{(1)} = 0.3$$

$$w_{2,2}^{(1)} = 0.4$$

$$w_{1,1}^{(2)} = 0.5$$

$$w_{2,1}^{(2)} = 0.6$$

$$b_1^{(1)} = 1$$

$$b_2^{(1)} = 1$$

$$b_1^{(2)} = 1$$



Cas plus concret d'un réseau de neurones simple

Passe forward

$$h_1^{(1)} = w_{1,1}^{(1)} \times x_1 + w_{2,1}^{(1)} \times x_2 + b_1^{(1)} \quad (35)$$

$$h_2^{(1)} = w_{1,2}^{(1)} \times x_1 + w_{2,2}^{(1)} \times x_2 + b_2^{(1)} \quad (36)$$

$$\hat{y} = w_{1,1}^{(2)} h_1^{(1)} + w_{2,1}^{(2)} h_2^{(1)} + b_1^{(2)} \quad (37)$$



Cas plus concret d'un réseau de neurones simple

En utilisant les données d'apprentissage on obtient:

$$h_1^{(1)}(0.5, 0.2) = 1.11 \quad (38)$$

$$h_1^{(1)}(1.1, 1) = 1.41 \quad (39)$$

$$h_2^{(1)}(0.5, 0.2) = 1.18 \quad (40)$$

$$h_2^{(1)}(1.1, 1) = 1.62 \quad (41)$$



Cas plus concret d'un réseau de neurones simple

- Dérivation de la fonction de coût pour chaque paramètre à estimer
- Les paramètres pour la couche de sortie et ceux des couches précédentes sont différents
- Faire une itération de mise à jour des paramètres